

The type parameter list introduces *type variables* that can be referenced in the rest of the type signature (exactly analogous to how variables introduced in the parameter list to a function can be referenced in the body of the function). In `findFirst`, the type variable `A` is referenced in two places: the elements of the array are required to have the type `A` (since it's an `Array[A]`), and the `p` function must accept a value of type `A` (since it's a function of type `A => Boolean`). The fact that the same type variable is referenced in both places in the type signature implies that the type must be the same for both arguments, and the compiler will enforce this fact anywhere we try to call `findFirst`. If we try to search for a `String` in an `Array[Int]`, for instance, we'll get a type mismatch error.



EXERCISE 2.2

Implement `isSorted`, which checks whether an `Array[A]` is sorted according to a given comparison function:

```
def isSorted[A](as: Array[A], ordered: (A,A) => Boolean): Boolean
```

2.5.2 Calling HOFs with anonymous functions

When using HOFs, it's often convenient to be able to call these functions with *anonymous functions* or *function literals*, rather than having to supply some existing named function. For instance, we can test the `findFirst` function in the REPL as follows:

```
scala> findFirst(Array(7, 9, 13), (x: Int) => x == 9)
res2: Int = 1
```

There is some new syntax here. The expression `Array(7, 9, 13)` is an *array literal*. It constructs a new array with three integers in it. Note the lack of a keyword like `new` to construct the array.

The syntax `(x: Int) => x == 9` is a *function literal* or *anonymous function*. Instead of defining this function as a method with a name, we can define it inline using this convenient syntax. This particular function takes one argument called `x` of type `Int`, and it returns a `Boolean` indicating whether `x` is equal to 9.

In general, the arguments to the function are declared to the left of the `=>` arrow, and we can then use them in the body of the function to the right of the arrow. For example, if we want to write an equality function that takes two integers and checks if they're equal to each other, we could write that like this:

```
scala> (x: Int, y: Int) => x == y
res3: (Int, Int) => Boolean = <function2>
```

The `<function2>` notation given by the REPL indicates that the value of `res3` is a function that takes two arguments. When the type of the function's inputs can be inferred